

SOEN 201 – UML Building Blocks (Weeks 4–5)

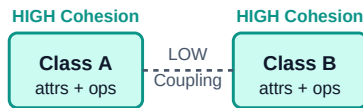
Kisii University | Clean Revision Notes

1. Unified Modelling Language (UML) & Class Principles

UML is a standard visual language for specifying, visualising, constructing and documenting the artefacts of software-intensive systems. It supplies the notation used throughout object-oriented analysis and design.

Principles of a well-designed class (from the lecture notes):

- **Attributes** – data held by the class; keep them focused on one clear responsibility.
- **Cohesion** – should be **high** inside a class. Related attributes and operations stay together → improves usability and reduces errors.
- **Coupling** – should be **low** between classes. Minimise dependencies so a change in one class does not cascade.



High cohesion inside each class; low coupling between classes.

2. Building Blocks of UML

UML is built from three fundamental kinds of building blocks:

1. **Elements** – the main “citizens” of a model (structural, behavioural, grouping, annotation).
2. **Relationships** – the ties that connect elements (dependency, association, generalisation, realisation).
3. **Diagrams** – graphical collections of elements and relationships that present different views of the system.

2.1 Structural vs Behavioural vs Grouping

- **Structural** – describes *what the system is* (classes, components, nodes ...). Does not change at run-time → **Static**.
- **Behavioural** – describes *what the system does* (interactions, state machines ...). Changes over time → **Dynamic**.
- **Grouping** – organises the model (packages). Purely organisational, development-time only → **Static**.

3. UML Elements – Quick Reference Table

Main UML elements from the handwritten notes. Purpose statements expanded slightly for revision clarity while remaining faithful to the lecturer’s wording.

Element	Type	Purpose (revision summary)	S/D
Class	Structural	Represents a set of objects that share the same attributes, operations, relationships and semantics.	Static
Interface	Structural	Collection of operations that specify a service; defines signatures only, not implementations.	Static
Collaboration	Struct. / Behav.	Defines an interaction of cooperating elements that together deliver a behaviour (often a design pattern).	Static
Component	Structural	Physical, replaceable part of a system that realises a set of interfaces; packages logical elements.	Static
Sequence	Behavioural	Shows interactions as a set of objects and the messages exchanged between them, ordered by time.	Dynamic
Interaction	Behavioural	Describes communication among objects (messages, action sequences, links) to achieve a purpose.	Dynamic
Package	Grouping	Primary mechanism for grouping and decomposing the model; conceptual, development-time only.	Static
Node	Structural	Physical run-time computational resource (memory & processing) on which components reside.	Static
Deployment	Structural	Shows how software artefacts are mapped onto physical nodes in the run-time environment.	Static

S/D = Static / Dynamic. Sequence & Interaction are the main dynamic (behavioural) elements listed in the notes.

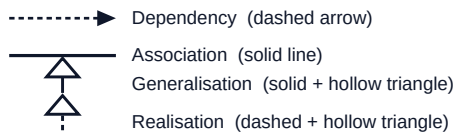
4. Relationships – Meaning & Notation

Four core relationship types connect UML elements. Meanings match the classic OO relationship concepts from the lecture.

Relationship	Meaning	Notation
Dependency / Inheritance*	Semantic relationship in which a change to one element (independent) may affect the meaning of the other (dependent).	----->
Association	Structural relationship describing a set of links between objects. Aggregation is a special whole-part form.	_____
Generalisation	Objects of a specialised element are substitutable for objects of a generalised element (is-a).	↑
Realisation	One element specifies a contract; another guarantees to carry it out (e.g. interface ↔ implementing class).	⬆

** In the handwritten notes “Dependency / Inheritance” appears as one entry; inheritance is most commonly drawn with the Generalisation notation.*

4.1 Visual Notation Summary



Remember the four arrow styles – they appear constantly in class and component diagrams.

5. Quick Revision Checklist

- UML = language for visualising, specifying, constructing and documenting software artefacts.
- Class design goal: high cohesion (related things together) + low coupling (few external dependencies).
- Three building blocks: Elements, Relationships, Diagrams.
- Structural elements = “nouns” (Class, Interface, Component, Node ...) → Static.
- Behavioural elements = “verbs” (Interaction, Sequence, State Machine ...) → Dynamic.
- Grouping element = Package (organisational only).
- Four relationships: Dependency (dashed), Association (solid), Generalisation (hollow ▴), Realisation (dashed + hollow ▴).
- Realisation = contract (interface) + guarantee (implementing class or component).
- Sequence emphasises chronological order of messages; Collaboration emphasises structural organisation of objects.